EXPERIMENT 12 B : IMPLEMENTATION OF A CHAT PROGRAM USING SOCKET PROGRAMMING

AIM:

To implement a simple client-server chat program using socket programming and demonstrate the exchange of messages over a TCP connection.

REQUIREMENTS:

Python 3 installed on two terminals or computers connected to the same network, and a text editor to create the server and client programs.

PROCEDURE:

1. Create a file named server.py and enter the server program given below.
2. Create a second file named client.py and enter the client program.
3. Run server.py first so that it starts listening for an incoming TCP connection.
4. Run client.py in another terminal on the same computer or on a permitted computer on the same network.
5. Enter messages in the client terminal. The server receives each message and sends a reply.
6. Type quit in the client terminal to end the conversation and close the connection.
7. Observe the messages printed by both the client and server and record the output.

PROGRAM:

Server program (server.py):

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(1)
    print("Chat server waiting for a connection...")
    conn, addr = server.accept()
    with conn:
        print("Connected by:", addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            message = data.decode("utf-8")
            print("Client:", message)
            if message.lower() == "quit":
                break
            reply = input("You: ")
            conn.sendall(reply.encode("utf-8"))
            if reply.lower() == "quit":
                break
    print("Chat ended.")
```

PROGRAM (CONTINUED):

Client program (client.py):

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.create_connection((HOST, PORT)) as client:
    print("Connected to chat server. Type 'quit' to exit.")
    while True:
        message = input("You: ")
        client.sendall(message.encode("utf-8"))
        if message.lower() == "quit":
            break
        data = client.recv(1024)
        if not data:
            break
        reply = data.decode("utf-8")
        print("Server:", reply)
        if reply.lower() == "quit":
            break
```

SAMPLE OUTPUT:

Server terminal:

```
Chat server waiting for a connection...
Connected by: ('127.0.0.1', 53124)
Client: Hello
You: Hi! How can I help?
Client: How are you?
You: I am fine.
Chat ended.
```

Client terminal:

```
Connected to chat server. Type 'quit' to exit.
You: Hello
Server: Hi! How can I help?
You: How are you?
Server: I am fine.
You: quit
```

OBSERVATION:

1. The server waits for a client and accepts the TCP connection.

2. Messages are encoded as bytes before transmission and decoded at the receiving end.

3. TCP provides a reliable, ordered stream of data between the client and server.

4. The quit command ends the chat session and closes the socket connection.

RESULT:

Thus, a simple chat program was implemented using Python socket programming. A TCP connection was established between the client and server, and messages were exchanged successfully.

Note: The sample outputs are illustrative. The programs use localhost (127.0.0.1); for two computers, use the server computer's LAN IP address and allow the chosen port through the local firewall as permitted.